

```
FOLLOWERS_FILE:followers.txt
FOLLOWS_FILE:following.txt
USERS_KEEP_FOLLOWING:
USERS_KEEP_UNMUTED:
USERS_KEEP_MUTED:
```

The files *already-followed.txt*, *followers.txt* and *following.txt* contain the respective twitter IDs. You must create these files — you can leave them empty, though. The rest of the fields may also be left empty.

```
# Program to retweet five latest tweets based on a hashtag
(#Serverless)
from TwitterFollowBot import TwitterBot

def retweet():
    # create an instance of the TwitterFollowBot
    # by default, the bot will look for a configuration
    # file called config.txt
    # in your current directory
    my_bot = TwitterBot()
    # autoretweets the 5(count)
    # latest tweets that matches the hashtag
    my_bot.auto_rt("#Serverless", count = 5)
    return {'message' : 'retweeted successfully'}

retweet()
```

That's the short and sweet code! Let's save it as *test.py* and run it. The output is shown in Figure 1.

When you execute the Python script, it connects to the Twitter API and retweets. The result throws up a warning indicating that the *followers.txt* isn't updated; you can just ignore that warning or update the files to get rid of the warning. The program displays the tweets that are retweeted, so we know it is working.

After we get the code working in our machine, it is time to deploy and execute it in the cloud using the serverless approach. We are going to use Apache OpenWhisk as the serverless platform. We will use IBM Bluemix as the cloud platform to deploy the OpenWhisk action(s).

Apache OpenWhisk

Apache OpenWhisk is an open source serverless cloud platform that executes functions in response to events at any scale.

OpenWhisk was started by IBM and is now incubated by Apache. Adobe is an important contributor to the project and has contributed the API Gateway. OpenWhisk executes functions (called actions) in response to events (called triggers). Since it is serverless, you just need to provide your code to be executed; specifically, you don't need to concern yourself with how to manage the life cycle or operations of the underlying containers that execute the code.

Figure 1: Output for *test.py*

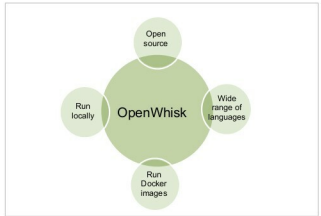


Figure 2: Key aspects of OpenWhisk

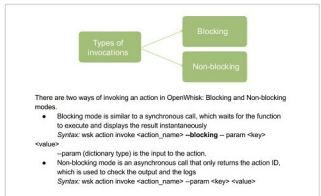


Figure 3: Types of action invocations

You can run OpenWhisk on your laptop, on-premise, or in the cloud. When running in the cloud, you could use a Platform-as-a-Service (PaaS) version of the OpenWhisk provided by IBM Bluemix or you can provision it yourself into Infrastructure-as-a-Service (IaaS) clouds, such as Bluemix, Amazon EC2, Microsoft Azure and Google Cloud Platform.

Here are some key aspects of OpenWhisk:

- It's open source. If you want, you can explore the code and tinker with it, and change it according to your requirements. Support for a wide range of programming languages including Node.js 6, Python 3, PHP 7.1 and Swift 3.1.1 is available.
- Actions (serverless functions) can also be custom executable programs packaged in a Docker container, i.e., you can run Docker images in OpenWhisk.
- You can run it locally! None of the major serverless platforms provide this feature. You can build and run actions locally on your own hardware behind your own firewall, and then deploy it to execute in the cloud.